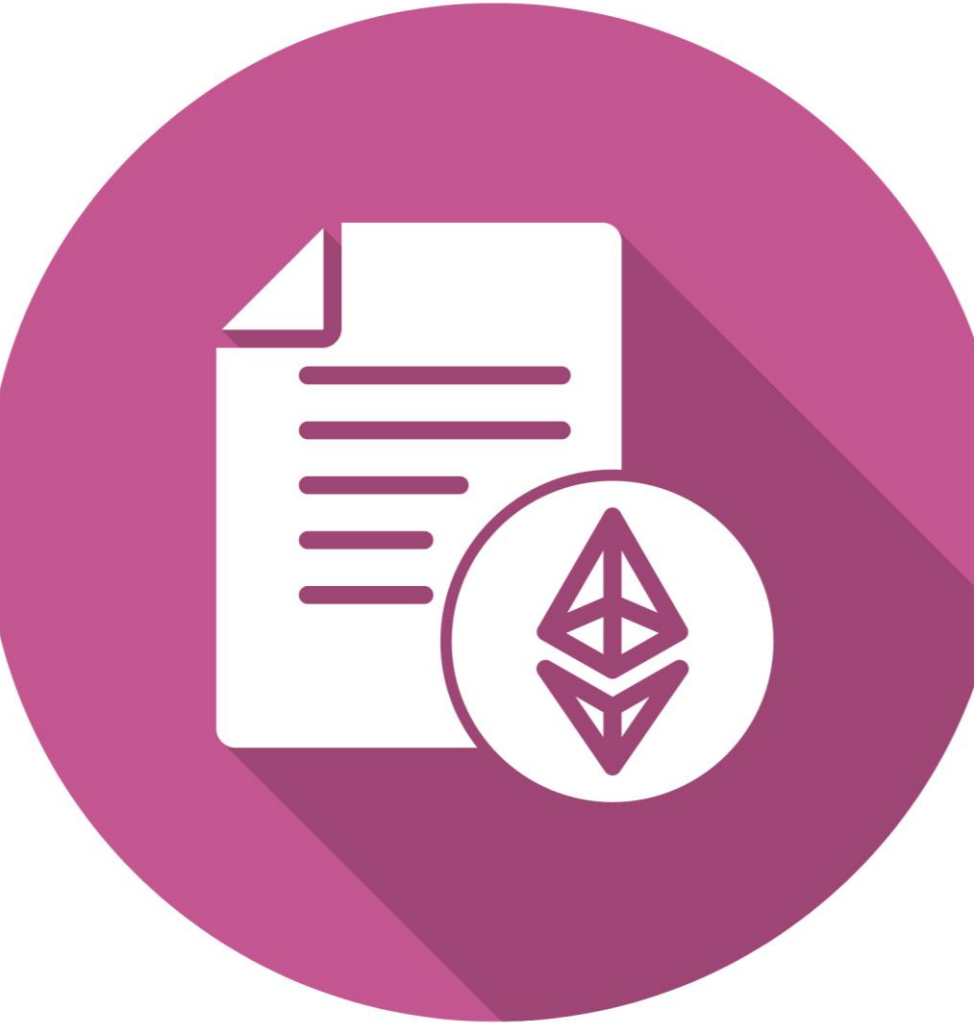
Code Eater



What is a smart contract ?

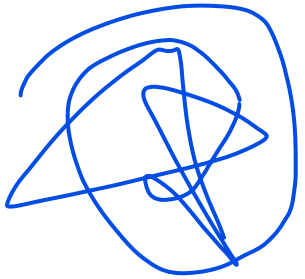
- Smart contracts are **simply programs stored on a blockchain .**

Smart Contract Application

A

B

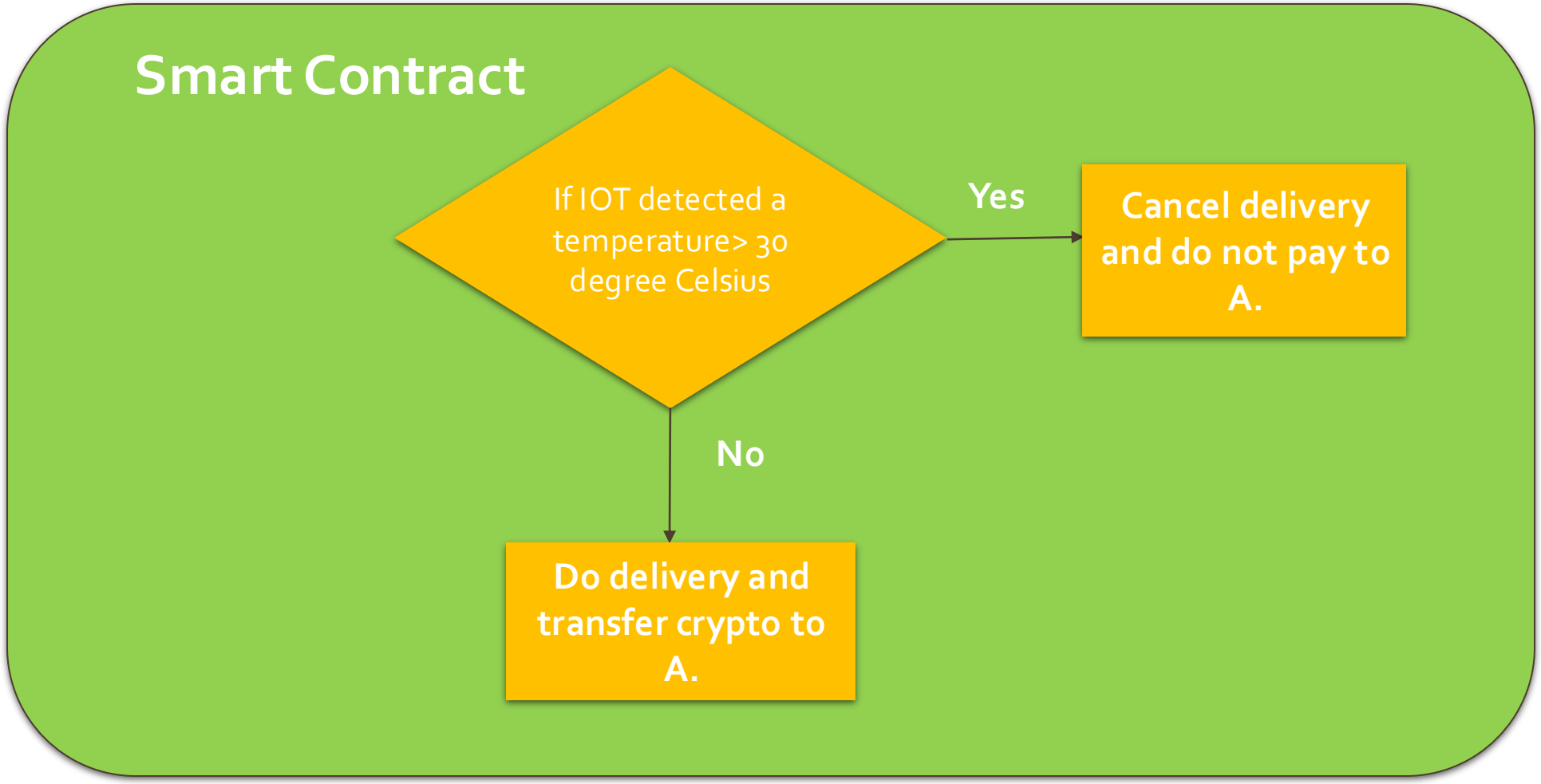
Smart Contract Application



A

B

Smart Contract Application



Note-Assuming optimum temperature <30 degree Celsius.

Smart Contract Features

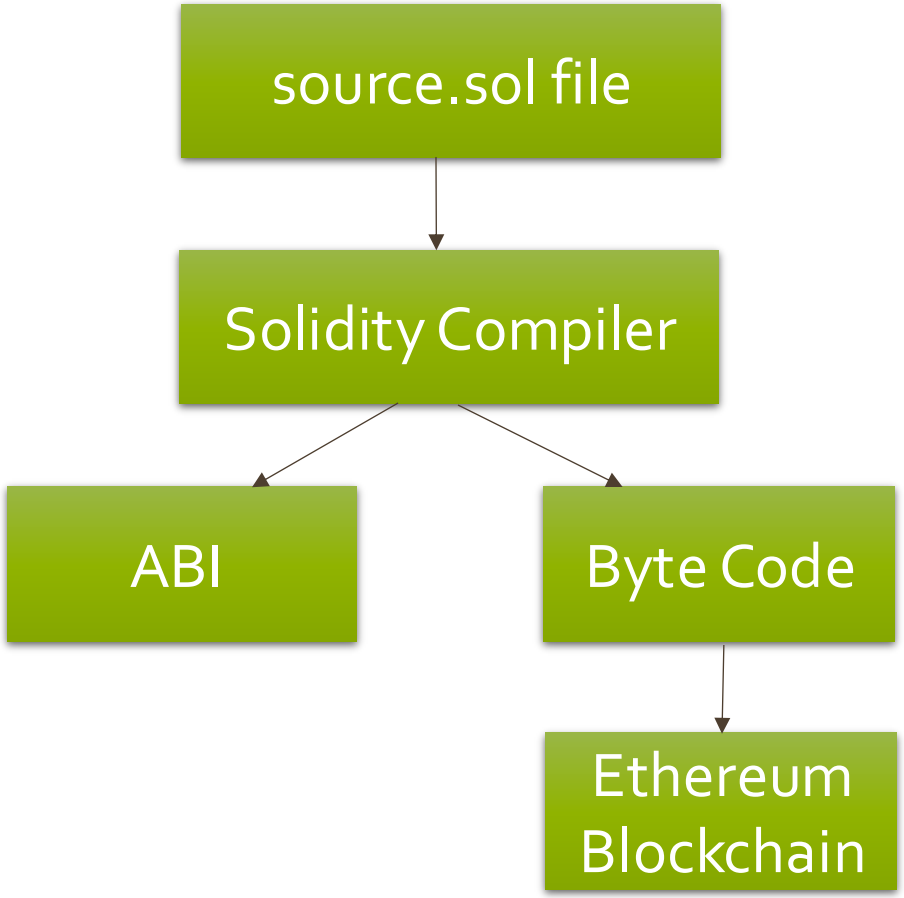
- Smart Contracts are immutable as they get stored on Blockchain.
- Smart contract contracts have their own accounts where it can store cryptocurrency.
- No human intervention is required for cryptocurrency transfer or receiving.

What is solidity?

- Solidity is an object-oriented programming language for implementing smart contracts for the ethereum blockchain.
- High-level statically typed programming language.
- Case sensitive.
- With Solidity you can create contracts for uses such as voting, crowdfunding, blind auctions, and multi-signature wallets.

Note – You should follow established development best-practices when writing your smart contracts.

Solidity Compilation Process



Solidity Compilation Process

- Contract bytecode is public in readable form.
- Contract doesn’t have to be public.
- Bytecode is immutable because it is getting stored on Blockchain.
- ABI act as a bridge between applications and smart contract.
- ABI and Bytecode cannot be generated without source code.

SPDX

- Trust in smart contracts can be better established if their source code is available. Since making source code available always touches on legal problems with regards to copyright, the Solidity compiler encourages the use of machine-readable SPDX license identifiers. Every source file should start with a comment indicating its license.

- Before publishing, consider adding a comment containing "**SPDX-License-Identifier: <SPDX-License>**" to each source file.

- Use "**SPDX-License-Identifier: UNLICENSED**" for non-open-source code.

- Please see SPDX & Etherscan for more information.

SPDX

SPDX License

File 1 of 7 : Token.sol

```
// contracts/Structs.sol
// SPDX-License-Identifier: Apache 2

pragma solidity ^0.8.0;

import "@openzeppelin/contracts/proxy/beacon/BeaconProxy.sol";

contract BridgeToken is BeaconProxy {
    constructor(address beacon, bytes memory data) BeaconProxy(beacon, data) {

    }
```

Wrapped BNB (Wormhole) (WBNB)

License : None

Contract Source Code (Solidity)

```
/**
 *Submitted for verification at Etherscan.io on 2020-09-16
*/

/**
 *Submitted for verification at Etherscan.io on 2020-09-15
*/

pragma solidity ^0.5.16;
pragma experimental ABIEncoderV2;

// From https://github.com/OpenZeppelin/openzeppelin-contracts/blob/master/contracts/math/Math.sol
// Subject to the MIT license.
```

UNI Token

Pragma

- In Solidity, pragma is a directive that specifies the compiler version or configurations that the source file should use. It helps ensure that the code is compiled with the appropriate version of the Solidity compiler, preventing potential issues that could arise from using different compiler versions. The most common use of pragma is to set the compiler version.

```
pragma solidity 0.8.0;
```

The directive pragma solidity 0.8.0; specifies that the Solidity compiler version must be exactly 0.8.0 to compile the smart contract.

```
pragma solidity ^0.8.0;
```

The caret ^ specifies that the compiler version should be **0.8.0 or any newer version that is backward compatible with 0.8.0**, but not 0.9.0 or later.

```
pragma solidity >=0.5.0 <0.9.0;
```

This means any version from 0.5.0 up to, but not including, 0.9.0 can be used.This range is useful for ensuring compatibility across multiple versions while avoiding breaking changes introduced in 0.9.0 and beyond.

State Variables

```
contract demo {
    uint public state_var;//state variable
}
```

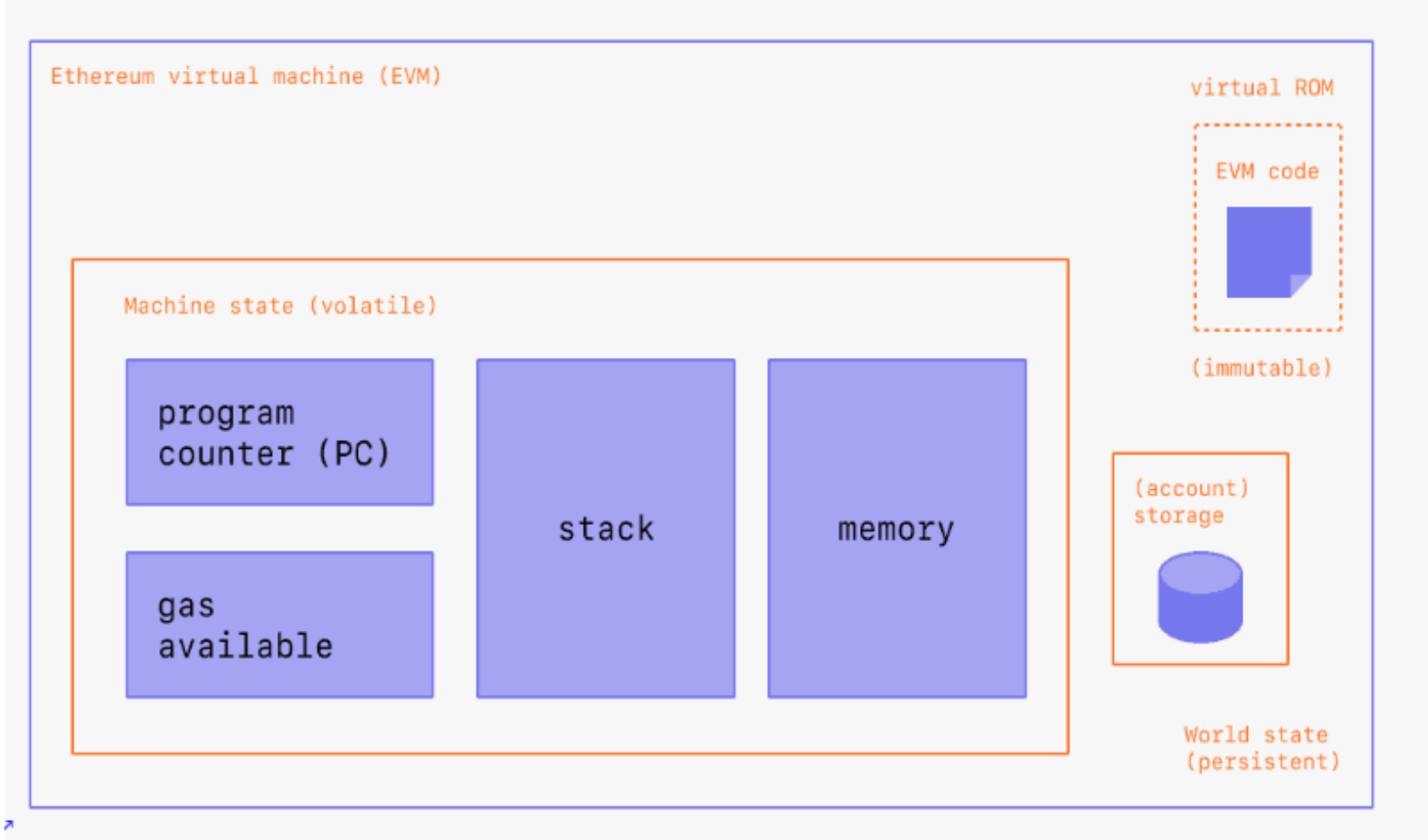
- Permanently stored in contract storage.
- Cost gas(expensive) .
- Reading of state variable is free but writing to it is costly.

Local Variables

```
function pureFunction() public pure {
    uint local_var; //local variable
    local_var = 1;
}
```

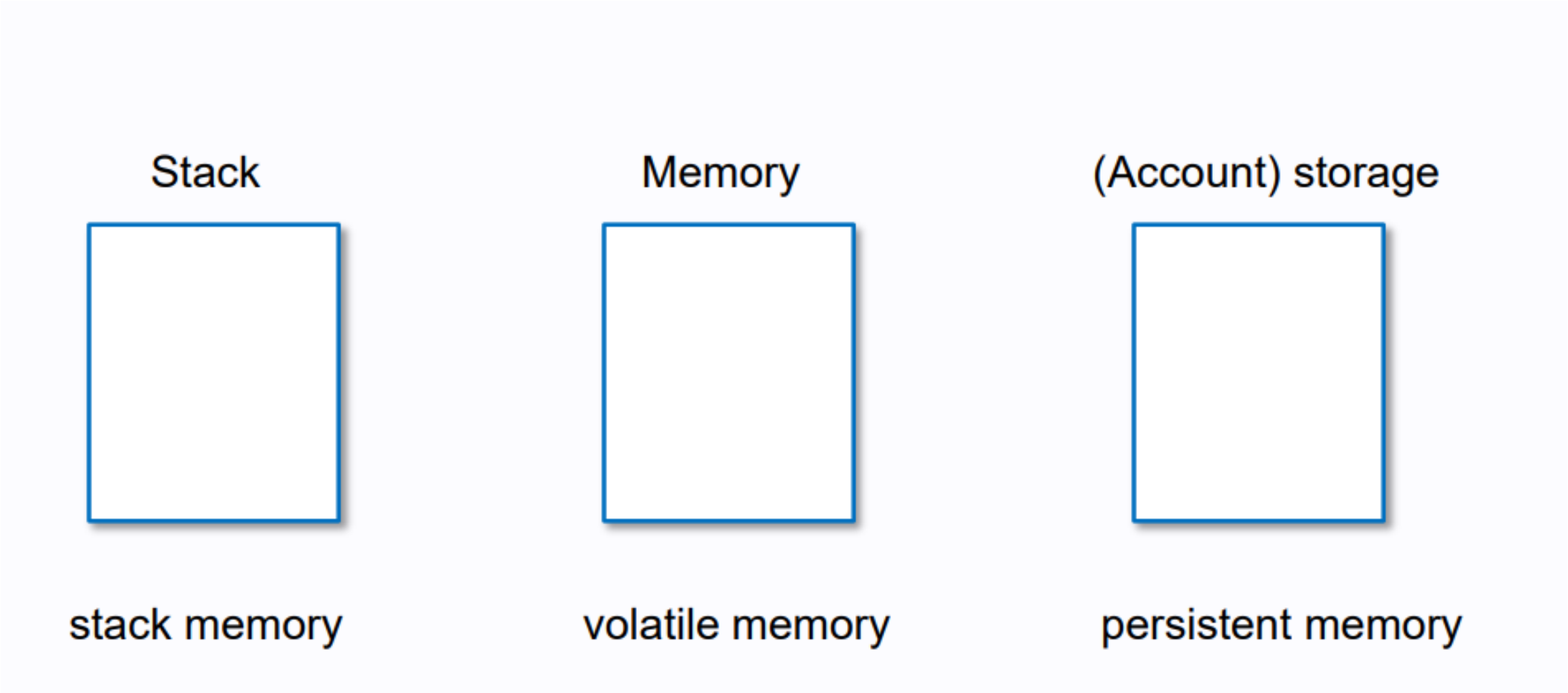
- Declared inside functions and are kept on the stack , not on storage.
- Don’t cost gas.

EVM


Ethereum virtual machine (EVM)
virtual ROM
EVM code
(immutable)
Machine state (volatile)
program counter (PC)
gas available
stack
memory
(account) storage
World state (persistent)

Code Eater

Storage Area



Functions

- When you declare a public state variable a getter function is automatically created.
- For public state variables a getter() function is automatically created.

View Vs Pure

Function Type	State Variable	
	Read	Write
View	✓	✗
Pure	✗	✗
		✓

Functions Example - 1

```
    uint public state_var;//state variable

    function setter() public { // we are writing to the state variable    22236 gas
        state_var = 2;
    }

    function getter() public view returns (uint) { // reading from the state variable    2437 g
        return state_var;
    }

    function pureFunction() public pure { // neither reading nor writing on the state variable
        uint local_var; //local variable
        local_var = 1;
    }
}
```

Code – Click Here

Function Example - 2

- Click Here

Constructor

```
contract demo {
    uint public state_var;//

    constructor(uint _val){
        state_var=_val;
    }
}
```

- Executed only once.
- You can create only one constructor and that is optional.
- A default constructor is created by the compiler if there is no explicitly defined constructor.

Basic Data Types

- Integer Data Type
- Bool Data Type
- Address Data Type
- Bytes Data Type

Integer Data Type

int

uint

Signed and Unsigned integers can be of various sizes.

int8 to int256

uint8 to uint256

int alias to int256

uint alias to uint256

By default int and uint are initialized to zero.

Overflow get detected at compile time.

Integer Data Type

Range	
int8 : - 128 to +127	**uint8** : 0 to 255
int16 : - 32768 to +32767	**uint16** : 0 to 65535
-2^(n-1) to 2^(n-1)-1	0 to 2^(n)-1

Bool Data Type

- bool public value = true;
- Bool data type value can be either true or false.
- By default value is false if not initialized.

Address Data Type

- address public addr = "0xBE4024Fa7461933F930DD3CEf5D1a01363E9f284"
- The address type is a 160-bit value that does not allow any arithmetic operations.

Bytes Data Type

- Bytes data type is used to store strings. Range - bytes1, bytes2, …..,bytes32.
- It stores characters.
- bytes1 public arr1="a";
- bytes2 public arr2="ab";
- bytes3 public arr3="abc";
- Everything that will be stored in the bytes array will be in hexadecimal number.
- arr3 will look this

61	62	63
0	1	2

- Click Here – Character To Hexadecimal Table
- Padding of 0 takes place if initialized characters are less than the byte size.

Bytes Data Type

```
contract demo {

    bytes2 public arr1 = "ab";

    function returnByte() public view returns (bytes1) {
        return arr1[0];
    }
}
```

Output - 0x61

Example - 1

Bytes Data Type

```
contract demo {

    bytes2 public arr = "ab";

    function returnArray() public view returns (bytes2) {
        return arr;
    }
}
```

Output - 0x6162

Example - 2

Conditionnels

```
function Conditionals(uint a) public pure returns (int) {
 if (a > 0) {
     return 0;
 } else if (a < 0) {
     return -1;
 } else {
     return 1;
 }
}
```

Code

Require

```
function someFunction(uint256 _value) public pure {    694 gas
 // Require that _value must be greater than 0
 require(_value > 0, "Value must be greater than zero");

 // Continue with the function's logic if the condition is met
 // ...
}
```

Code

Require

Example 2 - Code

Modifier

```
contract demo {
    modifier onlytrue {
        require(false == true, "_a is not equal to true");
        _;
    }

    function check1() public pure onlytrue returns (uint) {
        return 1;
    }

    function check2() public pure onlytrue returns (uint) {
        return 1;
    }

    function check3() public pure onlytrue returns (uint) {
        return 1;
    }
}
```

Code

Modifier

```
contract Example {

    // Modifier to check if a value is within a specified range
    modifier valueInRange(uint256 _value, uint256 _min, uint256 _max) {
        require(_value >= _min && _value <= _max, "Value out of range");
        _;
    }

    // Function that uses the valueInRange modifier
    function doSomething(uint256 _value) public valueInRange(_value, 10, 100) {
        // Function logic here
    }
}
```

Code

Loop

```
contract demo {
    function check1() public pure {    infinite gas
        for (uint i = 0; i < 7; i++) {
            // Loop body for 'for' loop (empty in this case)
        }

        while (true == true) {
            // Loop body for 'while' loop (empty in this case)
        }

        do {
            // Loop body for 'do-while' loop (empty in this case)
        } while (true == true);
    }
}
```

Code

Visibility

	PUBLIC	PRIVATE	INTERNAL	EXTERNAL
Outside World	✓			✓
Within Contract	✓	✓	✓	
Derived Contract	✓		✓	✓
Other Contracts	✓			✓

A **smart contract** is a **self-executing digital contract** where the terms are written in code and automatically enforced by a blockchain. It removes the need for intermediaries and ensures transparency, security, and efficiency in transactions. 🚀 🔗

1. **High-level** 🏗️ → A language that is closer to human language and abstracts low-level machine details, making it easier to read and write. Example: Python, Java.
2. **Statically Typed** 🔴 ✍️ → The type of a variable (like `int`, `string`) is **checked at compile-time**, meaning it must be declared before use and cannot change dynamically. Example: Java, C++.
3. **Programming Language** 💻 → A structured way to write instructions that a computer can understand and execute. Example: JavaScript, Python.

So, a **high-level statically typed programming language** is one that is **easy to read and write**, while ensuring **strict type enforcement at compile-time** for reliability and efficiency. 🚀

bytecode


```
6080604052348015600e575f80fd5b50610e1b8061001c5f395ff3fe6080604052348015610000f575f80fd5b506004361061005575f3560e01c8063028414f2146100
595780632ba53db2146100755780634418b79ab146100a75780636874404614610010d9578063a965b9881461010b575b5f80fd5b61007360048036038101906100 6e
9190610841565b61013b565b005b61008f60048036038101906100 8a91906108c9565b6101de565b60405161009e93929190610963565b60405180910390f35b61
00c160048036038101906100bc91906108c9565b61039a565b6040516100d093929190610963565b60405180910390f35b6100f360048036038101906100ee9190
6108c9565b61056c565b604051610102939291906109635565b604051809103 90f35b610125600480360381019061012091906108c9565b6106ac565b6040516101
3291906109a6565b60405180910390f35b5f604051806060016040528085815260200184815260200183815250908060018154018082558091505060019003905f
5260205f2090600302015f909190919091505f820151815f015560020820151816001019081610 19a9190610bb9565b506040820151816002019081610 1b0919061
0bb9565b50505060015f805490506101c49190610cb5565b60015f858152602001908152602001 5f2081905550505050565b5f6060805f805490508410610228576
040517f08c379a0000000000000000000000000000000000000000000000000000000008152600401 61021f90610d32565b60405180910390fd5b5f80858154811
061023c5761023b610d50565b5b905f5260205f2090600302016040518060600160405290815f820154815260200160018201805461026c906109ec565b80601f01
60208091040260200160405190810160405280929190818152602001828054610298906109ec565b80156102e35780601f106102ba576101008083540402835 29
1602001916102e3565b820191905f5260205f20905b815481529060010190602001808311610 2c657829003601f168201915b505050505081526020016002820 18
0546102fc906109ec565b80601f016020809104026020016040519081016040528092919081815260200182805461032890 6109ec565b80156103735780601f106
1034a576101008083540402835291602001916103735 65b820191905f5260205f20905b815481529060010190602001808311610356578290036 01f168201915b5
05050505081526050905080 5f015181602001518260400151935093509350509193909250565b5f6060805f60015f868152602001908152602001 5f205490505f80
54905081106103f9576040517f08c379a0000000000000000000000000000000000000000000000000000000008152600401 03f090610dc7565b604051809103
90fd5b5f808281548110610 40d5761040c610d50565b5b905f5260205f20906003020160405180606001604052908 15f8201548152602001600182018054610 43d9
06109ec565b80601f016020809104026020016040519081016040528092919081815260200182805461046990 6109ec565b80156104b45780601f1061048b57610
100808354040283529160200191610544565b820191905f5260205f20905b815481529060010190602001808311610497578290036 01f168201915b50505050508
152602001600282018054610 4cd906109ec565b80601f016020809104026020016040519081016040528092919081815260200182805461046 04f9906109ec565b801
56105445780601f1061051b576101008083540402835291602001916105445 65b820191905f5260205f20905b815481529060010190602001808311610527578 29
003601f168201915b505050505081525050905080 5f015181602001518260400151945094509450505091939092505 65b5f818154811061057a575f80fd5b905f52
60205f2090600302015f91509050805f015490806001018054610 59f906109ec565b80601f016020809104026020016040519081016040528092919081815260200
18280546105cb906109ec565b80156106165780601f106105ed576101008083540402835291602001916106165 65b820191905f5260205f20905b8154815290600
10190602001808311610 5f957829003601f168201915b505050505090806002018054610 62b906109ec565b80601f016020809104026020016040519081016040 5
28092919081815260200182805461065790 6109ec565b80156106a25780601f106106795761010080835404028352916020019161 06a2565b820191905f5260205
f20905b815481529060010190602001808311610685578290036 01f168201915b505050505090508 3565b6001602052805f5260405f205f91509050548 1565b5f60
4051905090565b5f80fd5b5f80fd5b5f80fd5b5f8190509190505 65b6106e4816106d2565b81146106ee575f80fd5b50565b5f813590506106ff816106db565b929150505 65b5f
80fd5b5f80fd5b5f601f19601f830116905091905056 5b7f4e487b7100000000000000000000000000000000000000000000000000000000005f52604160045260245ff
d5b6107538261070d565b810181811067ffffffffffffffff8211171561077 25761077161071d565b5b806040525050505 65b5f6107846106c1565b90506107908282610
74a565b919050565b5f67ffffffffffffffff8211156107af576107ae61071d565b5b6107b88261070d565b905060208101905091905056 5b828183375f83830152505050
565b5f6107e56107e084610795565b61077b565b905082815260208101848484011115610801576108006107095 65b5b1080c8482856107c5565b509392505050
565b5f82601f830112610828576108276107050 5565b5b813561083884826020860161107d3565b915050929150505 65b5f805f606084860312156108585761085761
06ca565b5b5f6108658682870161 06f1565b935050602084013567ffffffffffffffff8111156108865761088856106ce565b6108928682870161081456 5b92505060408
4013567ffffffffffffffff8111156108b3576108b26106ce565b5b6108bf8682870161081456 5b915050925092509256 5b5f6020828403121561086de576108dd6106ca56
5b5b5f6108eb848285016106f1565b915050929150505 65b6108fd816106d2565b82525050565b5f8151905091905056 5b5f82825260208201905092915050565b8
281835e5f8383015250505056 5b5f610935826109035 65b61093f818561090d565b935061094f818560208601610 91d565b6109588161070d565b840191505092 9
15050565b5f606082019050610977 65f8301866108f4565b818103602083015261098881856109 2b565b905081810360408301526109 9c818461092b565b9050949
350505050565b5f602082019050610 9b95f8301846108f4565b92915050565b7f4e487b7100000000000000000000000000000000000000000000000000000000005f
52602200452602 45ffd5b5f6002820490506001821680610a0357607f821691505b602082108103610a1657610a156109bf565b5b50919050565b5f82810501 5f52
60205f2090509190505 65b5f6020601f830104905091905056 5b5f82821b905092915050565b5f60088302610a787ffffffffffffffffffffffffffffffffffffffffffffffffffffffff
82610a3d565b610a828683610a3d565b9550801984169350808616841792505050939250505 65b5f819050919050565b5f610abd610ab8610ab3846106d2565b
610a9a565b6106d2565b905091905056 5b5f819050919050565b5f610ad683610aa3565b610ad683610aa2826210ac4565b848454610a49565b82555050505056 5b5f905
65b610afe610af2565b610b09818484610acd565b505050565b5b81811015610b2c57610b215f82610af6565b600181019050610b0f565b5050565b601f821115610
b7157610b4281610a1c565b610b4b84610a2e565b810160208510156 10b5a578190505b610b6e610b6685610a2e565b830182610b0e565b50505b505050565b5f8
2821c905092915050565b5f610b915f19846008026 10b76565b198083169150509291505056 5b5f610ba98383610b82565b5f1508260020282179050929150505 65
b610bc282610903565b67ffffffffffffffff811115610bdb57610bda61071d565b5b610be582546109ec565b610bf0828285610b30565b5f60209050601f8311600181146
10c21575f8415610c0f5782870151905056 5b610c198582610b9e565b865550610c80565b601f198416610c2f86610a1c565b5f5b82811015610c5657848901518255 6
00182019150602085019450602081019050610c31565b86831015610c735784890151610c6f601f891682610b82565b8355505b600160028802188555050505b5
05050505050565b7f4e487b7100000000000000000000000000000000000000000000000000000000005f52601160045260245ffd5b5f610cbf826106d2565b915061
0cca836106d2565b925082820390508181111 5610ce257610ce1610c88565b5b92915050565b7f496e646578206f7574206f6620626f756e64730000000000000000
0000000005f82015250565b5f610d1c601383610d90d565b9150610d2782610ce8565b602082019050919050565b5f602082019050818103 5f830152610d4981610
d10565b9050919050565b7f4e487b7100000000000000000000000000000000000000000000000000000000005f52603260045260245ffd5b7f426f6f6b206e6f74206
66f756e640000000000000000000000000000000000000005f82015250565b5f610db1600e8361090d565b9150610dbc82610d7d565b602082019050919050565b5f60
208201905081810 35f830152610dde81610da5565b905091905056fea2646970667358221220 35b91446bc3d98eb650ad60ff9a719c5edefcc25fe22d40ca331e3c53a
fa083964736f6c634300081a0033
```

SPDX License in Solidity

SPDX (Software Package Data Exchange) License Identifier is a short, machine-readable way to specify the license under which a Solidity smart contract is distributed. It appears at the top of Solidity files as a comment.

Why is it used?

- Helps with **open-source compliance** 📜
- Makes licensing **clear and standardized** ✅
- Required by some compilers to avoid warnings ⚠️

Example in Solidity:

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
```

- 🔹 `"MIT"` means the contract follows the **MIT License** (permissive, widely used).
- 🔹 Other common licenses: `"GPL-3.0"`, `"Apache-2.0"`, `"Unlicense"`, etc.

It's **optional** but recommended for transparency in open-source projects! 🚀

If your **Solidity contract is not open-source**, you can use:

```solidity
// SPDX-License-Identifier: UNLICENSED
```

✅ This means you **do not grant any license** for others to use, modify, or distribute your code.

🔹 **Why use it?**

- Avoids compiler warnings ⚠️
- Clearly states that the contract is **not open-source** 🚫
- Protects your intellectual property 🛡️

Even if **not open-source**, adding an SPDX identifier is a good practice! ✅

STATE-VARIABLES

EXAMPLE-1

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.8.2 <0.9.0;

contract demo {
    uint256 public state_var; //state variable

    function setter() public {
        // we are writing to the state variable
        state_var = 2;
    }

    function getter() public view returns (uint256) {
        // reading from the state variable
        return state_var;
    }

    function pureFunction() public pure returns (uint256) {
        // neither reading nor writing on the state variable
        uint256 local_var; //local variable
        local_var = 1;
        return local_var;
    }
}
```

EXAMPLE-2

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.8.2 <0.9.0;

contract Demo {
    //this is the place to write code
    int256 public counter = 10; //state variable

    function writeStateVariable() public {
```

```
        counter = 20; //writing the state variable
    }

    function readAndWriteStateVariable() public returns (int256) {
        counter = 100; //writing the state variable
        return counter; //reading the state variable
    }

    function readStateVariable() public view returns (int256) {
        // counter = 100; //writing the state variable
        return counter; //reading the state variable
    }

    function returnLocalVariable() public pure returns (bool) {
        bool value = true;
        return value;
    }

    function readStateVariableLocalVariable() public view returns (bool) {
        int256 localVar = counter;
        bool value = true;
        return value;
    }

    function writeStateVariableLocalVariable() public returns (bool) {
        counter = 20;
        bool value = true;
        return value;
    }
}
```

Solidity Storage Types (Simple Explanation)

1️⃣ **Stack (Temporary, Fastest, Small Data)**

- Stores small values like `uint`, `bool`, `address` inside functions.
- Very fast but has a **size limit (1024 slots).**
- Gets **cleared after function execution.**
- **No gas cost** (CPU handles it).

2️⃣ **Memory (Temporary, Used in Functions)**

- Like **RAM**, holds **temporary** data (arrays, structs, strings) inside functions.
- Data **disappears after the function ends.**
- **Moderate gas cost** (reading is cheap, writing costs gas).
- Used for **dynamic data types.**

3️⃣ **Storage (Permanent, Expensive, Contract-Level)**

- Stores **contract state variables** permanently on the blockchain.
- **High gas cost** (modifying storage changes blockchain state).
- Data **remains even after function execution.**
- Used for **important contract data** (mappings, large arrays).

💡 **Summary:** Stack is **fastest but limited**, Memory is **temporary**, and Storage is **permanent but costly**! 🚀

EXAMPLE-3

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.8.2 <0.9.0;

contract Demo {
//this is the place to write code
 int public counter=10;//state variable

 function writeStateVariable() public {
    counter=20;//writing the state variable
 }
```

```
 function readAndWriteStateVariable() public returns(int){
    counter=100; //writing the state variable
    return counter;//reading the state variable
 }
  function readStateVariable() public returns(int){
    counter=100; //writing the state variable
    return counter;//reading the state variable
 }

 function returnLocalVariable() public pure returns(bool){
    bool value=true;
    return value;
 }
  function readStateVariableLocalVariable() public view returns(bool){
    int localVar = counter;
    bool value=true;
    return value;
 }
   function writeStateVariableLocalVariable() public returns(bool){
    counter=20;
    bool value=true;
    return value;
 }

}
```

CONSTRUCTOR EXAMPLE

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.8.2 <0.9.0;

contract Student {
    string public name;
    uint256 public age;
    uint256 public id;

    // Constructor to initialize the contract with a student's details
    constructor(
        string memory _name,
        uint256 _age,
        uint256 _id
    ) {
        name = _name;
```

```
        age = _age;
        id = _id;
    }

    // Function to update the student's details
    function updateDetails(
        string memory _name,
        uint256 _age,
        uint256 _id
    ) public {
        name = _name;
        age = _age;
        id = _id;
    }

    // Function to get the student's details
    function getDetails()
        public
        view
        returns (
            string memory,
            uint256,
            uint256
        )
    {
        return (name, age, id);
    }
}
```

INTEGER DATA-TYPE

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.8.2 <0.9.0;

contract IntegerExample {
    // State variables to store signed integers
    int256 public smallNumber; // 256-bit signed integer
    int8 public tinyNumber; // 8-bit signed integer
```

```
    // Constructor to initialize the state variables
    constructor(int256 _smallNumber, int8 _tinyNumber) {
        smallNumber = _smallNumber;
        tinyNumber = _tinyNumber;
    }

    // Function to set new values for the state variables
    function setNumbers(int256 _smallNumber, int8 _tinyNumber) public {
        smallNumber = _smallNumber;
        tinyNumber = _tinyNumber;
    }

    // Function to get the values of the state variables
    function getNumbers() public view returns (int256, int8) {
        return (smallNumber, tinyNumber);
    }

    // Function to demonstrate arithmetic operations with integers
    function addNumbers(int256 a, int256 b) public pure returns (int256) {
        return a + b;
    }

    function subtractNumbers(int256 a, int256 b) public pure returns
(int256) {
        return a - b;
    }

    function multiplyNumbers(int256 a, int256 b) public pure returns
(int256) {
        return a * b;
    }

    function divideNumbers(int256 a, int256 b) public pure returns
(int256) {
        require(b != 0, "Division by zero is not allowed");
        return a / b;
    }
}
```

Understanding `int8`:

- `int8` is a **signed 8-bit integer** (can store both positive and negative numbers).
- **8 bits** means we have 2^8 **(256) possible values**, but these are split between **negative and positive** numbers.
- The range of `int8` is:

$$-2^7 \text{ to } (2^7 - 1)$$

$$-128 \text{ to } 127$$

(Since one bit is used for the sign, we get `2⁷` values for negative and `2⁷ - 1` values for positive numbers.)

What about `uint8`?

If you use `uint8` (unsigned 8-bit integer):

- No negative numbers
- Stores values from `0` to `2⁸ - 1` (`0` to `255`)

Final Answer:

❌ **Wrong:** *"int8 means we can store `2⁸ = 256` numbers."*
✅ **Correct:** *"int8 can store values from `-128` to `127` (256 values in total)."* 🚀

ADDRESSDATA-TYPE

```
pragma solidity >=0.8.2 <0.9.0;

contract SimpleAddressExample {
    address public owner; // State variable to store the address of the
contract owner
    address public otherAddress; // State variable to store another
address

    // Constructor to initialize the contract with the owner's address
    constructor() {
        owner = msg.sender; // Set the owner to the address that deploys
the contract
```

```
    }

    // Function to set another address
    function setOtherAddress(address _otherAddress) public {
        otherAddress = _otherAddress;
    }

    // Function to get the owner's address
    function getOwner() public view returns (address) {
        return owner;
    }

    // Function to get the other address
    function getOtherAddress() public view returns (address) {
        return otherAddress;
    }
}
```

BYTES DATA-TYPE

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.8.2 <0.9.0;

contract demo {
    bytes3 public arr1 = "ab";

    function returnArray() public view returns (bytes1) {
        return arr1[0];
    }
    function returnwholeArray() public view returns (bytes3) {
        return arr1;
    }
}
```

CONDITIONS

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.8.2 <0.9.0;
```

```
contract demo {
    function Conditionals(int256 a) public pure returns (int256) {
        if (a > 0) {
            return a;
        } else if (a < 0) {
            return -1;
        } else {
            return 1;
        }
    }
}
```

REQUIRE

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.8.2 <0.9.0;

contract demo {
    int256 public num;

    function requireCheck(int256 a) public {
        num = 100;
        require(a > 0, "a is not greater than zero");
        num = a;
    }

    function conditionals(int256 b) public returns (string memory) {
        num = 200;
        if (b > 0) {
            num = b;
        } else {
            return "b is not greater than zero";
        }
    }
}

//error throw in require check
```

```
//statement return in conditionals

//function require check reverse ho jayega
```

MODIFIER
EXAMPLE-1

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.8.2 <0.9.0;

contract demo {
    modifier onlytrue() {
        require(false == true, "_a is not equal to true");
        _;
    }

    function check1() public pure onlytrue returns (uint256) {
        return 1;
    }

    function check2() public pure onlytrue returns (uint256) {
        return 1;
    }

    function check3() public pure onlytrue returns (uint256) {
        return 1;
    }
}
```

EXAMPLE-2

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.8.2 <0.9.0;

contract Example {
    // Modifier to check if a value is within a specified range
    modifier valueInRange(
        uint256 _value,
```

```
        uint256 _min,
        uint256 _max
    ) {
        require(_value >= _min && _value <= _max, "Value out of range");
        _;
    }

    // Function that uses the valueInRange modifier
    function doSomething(uint256 _value)
        public
        pure
        valueInRange(_value, 10, 100)
        returns (string memory)
    {
        return "good";
    }
}
```

LOOPS

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract LoopExamples {
    // For loop example
    function forLoopExample(uint256 n) public pure returns (uint256) {
        uint256 sum = 0;
        for (uint256 i = 0; i <= n; i++) {
            sum = sum + i;
        }
        return sum;
    }

    // While loop example
    function whileLoopExample(uint256 n) public pure returns (uint256) {
        uint256 sum = 0;
        uint256 i = 0;
        while (i <= n) {
            sum += i;
```

```
            i++;
        }
        return sum;
    }

    // Do-while loop example
    function doWhileLoopExample(uint256 n) public pure returns (uint256) {
        uint256 sum = 0;
        uint256 i = 0;
        if (n > 0) {
            // Ensuring the do-while loop runs at least once if n is
greater than 0
            do {
                sum = sum + i;
                i = i + 1;
            } while (i <= n);
        }
        return sum;
    }
}
```

Visibity	KAKA	M	LOAN	~~B~~ ~~Y~~ ~~A~~
	public	private	internal	externe
outside world	✓			✓
within contract	✓	✓	✓	
derived contract	✓		✓	✓
other contract	✓			✓

```
// contracts/GLDToken.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract demo {
    function f1() public pure returns (uint256) {
        // f2();
        return 1;
    }
```

```
    function f2() private pure returns (uint256) {
        return 2;
    }

    function f3() internal pure returns (uint256) {
        return 3;
    }

    function f4() external pure returns (uint256) {
        return 4;
    }
}

contract otherContract {
    demo obj = new demo(); //creating object
    uint256 public y = obj.f4();
}

contract child is demo {
    uint256 public x = f1();
}
```